PROJECT MANAGEMENT DOCUMENT

ExpenseEase Project Plan

Shared Expense Management System

Course	Software Engineering and Project Management (CSN15101)
Instructor	Dr. Satarupa Chakrabarti
Development model	Iterative and incremental delivery, supported by early workflow prototyping
Project repository	github.com/shreesriv12/ExpenseEase

Project status at a glance

4 / 4	179	PASS
planned increments complete	automated tests passed	lint and production build

1. Project overview

ExpenseEase is a full-stack application for recording expenses shared by group members, calculating each member's net balance, simplifying debts, and recording settlements. It aims to make shared spending transparent and repeatable by using deterministic calculations and an auditable activity history. The plan organizes the work into small, verifiable increments so that core workflows can be integrated and tested as they are developed.

2. Objectives

- Provide secure registration and authenticated group membership workflows.
- Support expense creation, editing, and deletion with equal, exact-amount, and percentage splits.
- Compute deterministic balances, present dashboard summaries, and support settlement recording and activity tracking.
- Maintain quality through automated tests, lint checks, production builds, and containerized local startup.

3. Scope baseline

Included: authentication, group membership, expense CRUD, split validation, balances, debt simplification, settlements, activity history, dashboard summaries, PostgreSQL/Prisma integration, and Docker Compose startup. Engineering principle: split and balance calculations are kept deterministic and tested independently of UI and data-access concerns.

4. Lifecycle and increment plan

The team used an iterative, incremental SDLC. Each increment moved from requirements and design into implementation, integration, and test feedback before the next major workflow was added. Early workflow prototypes helped clarify interactions such as group membership, expense splitting, and settlement entry.

Stage / increment	Deliverable and scope	Status	Evidence / exit condition
Requirements and design	Problem definition, scope, user stories, architecture, database design, and UI wireframes.	**COMPLETE**	Requirements and design artefacts are prepared.
Increment 1	Authentication and group membership workflow.	**COMPLETE**	Protected API routes and registration/group interface.
Increment 2	Expense CRUD with equal, exact-amount, and percentage splits.	**COMPLETE**	Split services, expense interface, and automated tests.
Increment 3	Balances, debt simplification, settlements, dashboard, and activity history.	**COMPLETE**	Services, UI workflows, and integration tests.
Increment 4	Docker/PostgreSQL setup, quality checks, and technical documentation.	**COMPLETE**	Container startup, 179 tests, lint, production build, and test evidence.

5. Dependency sequence

01 Requirements / design	02 Auth / groups	03 Expenses / splits	04 Balances / settlement	05 Quality / container
Define users, rules, data, and interfaces.	Establish identity and group access.	Build on authenticated group context.	Consume recorded expenses and shares.	Verify integrated behavior and startup.

Completion criteria

An increment is considered complete when its main user workflow is implemented, the related service or API behavior is integrated, automated tests cover important success and validation paths, and the existing application continues to pass the relevant quality checks. The current status summary marks all four increments complete.

6. Work breakdown and team plan

The work is split into requirements/design, core feature implementation, integration, and quality activities. The project uses shared ownership across frontend and backend work, with an equal planned contribution of 25% per team member.

Work package	Activities	Expected output	Needs
A. Requirements and analysis	Define problem, actors, scope, functional and non-functional requirements, and user stories.	Reviewed requirements baseline and prioritized workflows.	Start
B. Architecture and data design	Separate UI, API, service logic, and persistence; design entities and interfaces.	Architecture and database model.	A
C. Authentication and groups	Implement registration/login, protected routes, and group membership checks.	Authenticated group workflows.	A-B
D. Expenses and split rules	Implement expense CRUD and equal, exact, and percentage split validation.	Expense workflows with tested calculations.	C
E. Balances and settlements	Calculate net balances, simplify debts, record settlements, and show activity/dashboard data.	End-to-end balance and settlement workflows.	D
F. Integration and quality	Configure Docker/PostgreSQL, run tests, lint, build, and address integration defects.	Reproducible startup and verified quality checks.	C-E

7. Contribution allocation

Team member	Planned contribution	Responsibility framing
Shreeya Srivastava 20243267	**25%**	Shared frontend and backend project work.
Shraddha Sharma 20243266	**25%**	Shared frontend and backend project work.
Rudransh Pratap Singh 20243243	**25%**	Shared frontend and backend project work.
Sachit Jain 20243245	**25%**	Shared frontend and backend project work.

The supplied plan defines equal 25% contribution across the team and does not identify separate member-specific task percentages. The table therefore records the shared allocation without inventing individual ownership claims.

8. Technical baseline

Layer	Technology / design choice
Frontend	React, Vite, and Axios for interactive user workflows.
Backend	Node.js and Express API, with request validation using Zod.
Data	PostgreSQL with Prisma ORM; Docker Compose supports local startup and service dependencies.
Security	bcrypt password hashing, JWT-protected routes, and group-membership authorization.
Verification	Vitest and Supertest automated tests, lint checks, and production build.

9. Quality plan and verification

Quality activities run alongside implementation rather than being deferred until the end. Core calculations are isolated in service logic so they can be tested independently, while integration tests exercise API behavior with a test database.

Quality activity	Purpose / expected result	Current evidence
Unit tests	Verify split calculations, rounding/remainder behavior, validators, and balance invariants.	Included in automated test suite.
Integration tests	Verify end-to-end API workflows, persistence, settlement effects, and activity records.	Integration tests reported as complete.
Static quality checks	Catch lint issues before production build and integrated verification.	Lint passed.
Production build	Verify that the application builds with production settings.	Production build passed.
Container startup	Start database, wait for health, apply migrations, and then start the app.	Docker startup evidence is part of Increment 4.

Verification summary

179	157	22	30 Sep 2026
total automated tests passed	server tests	client tests	recorded verification date

10. Risk register and mitigation

Risk	Potential effect	Mitigation / control
Rounding or split-validation defects	Incorrect shares or net balances.	Use integer paise, deterministic remainder allocation, input validation, and invariant tests proving shares sum to the stored total.
Changing requirements	Rework across UI, services, and data model.	Use incremental delivery; review impact before changing requirements and keep related design and tests aligned.
Team availability	Uneven progress or delayed integration.	Keep work in small increments, communicate dependencies, and use shared module knowledge.
Integration failures	Defects discovered late in a workflow.	Integrate progressively and run tests, lint, and build checks after changes.
Authorization or data-scope defects	Users may access data outside their groups.	Require valid JWTs on protected routes and enforce membership-based scoping; add negative authorization tests.

11. Monitoring and change control

Progress is monitored against increment exit criteria, test outcomes, lint, build status, and integration behavior. Changes that affect requirements or data rules should record the reason and impact, with corresponding updates to the requirements, design, and tests. The main branch should remain buildable and testable.

Current position: the supplied project status reports all planned increments complete, 179 automated tests passing, lint passing, and the production build passing. No additional engineering milestone was identified as pending in the provided plan.